

LIVRO DE ATIVIDADES DE NODE



1. Fundamentos do Node.js

Responda com clareza e complete as linhas com sua própria explicação. Sempre que possível, use exemplos práticos.

Explique com suas palavras o que é o Node.js e por que ele não é considerado uma linguagem de programação.

Qual é a diferença entre JavaScript executado no navegador e JavaScript executado com Node.js?

Descreva o que significa dizer que o Node.js usa uma arquitetura orientada a eventos.

O que é o event loop e qual o papel dele no processamento de tarefas assíncronas?

2. Módulos, NPM e Estrutura de Projeto

Responda com clareza e complete as linhas com sua própria explicação. Sempre que possível, use exemplos práticos.

Qual é a função do arquivo package.json dentro de um projeto Node?

Explique a diferença entre dependências e devDependencies.

Para que servem module.exports e require no sistema CommonJS?

Qual a diferença prática entre importar um módulo nativo, um arquivo local e um pacote instalado via npm?

3. Servidores, APIs e Backend

Responda com clareza e complete as linhas com sua própria explicação. Sempre que possível, use exemplos práticos.

O que o módulo http permite fazer em uma aplicação Node.js?

Explique a diferença entre rota, controlador e serviço em uma API organizada.

Por que Express se tornou uma biblioteca tão usada em projetos Node?

Descreva o fluxo de uma requisição HTTP desde a chegada ao servidor até o envio da resposta.

4. Assincronicidade, Banco de Dados e Boas Práticas

Responda com clareza e complete as linhas com sua própria explicação. Sempre que possível, use exemplos práticos.

Explique a diferença entre callback, Promise e async/await.

Por que operações com banco de dados geralmente precisam ser assíncronas?

Cite três boas práticas importantes para projetos backend em Node.js.

Descreva um pequeno backend que você conseguiria construir com Node hoje e quais partes principais ele teria.

[illegible]

5. Desafio Prático

Leia a proposta abaixo e esboce a solução. O objetivo é conectar teoria e prática.

Você foi contratado para criar uma pequena API para gerenciar tarefas. Essa API deve permitir cadastrar tarefas, listar tarefas, atualizar o status e remover tarefas. Responda às questões abaixo.

Quais rotas HTTP você criaria para essa API?

Que dados uma tarefa deveria ter no mínimo?

Como você separaria responsabilidades entre rotas, controllers e services?

Onde entrariam validações e tratamento de erros?

Se esse projeto usasse banco de dados, como seria o fluxo entre API e banco?

Bônus

Escreva um pequeno trecho de pseudocódigo ou código Node.js mostrando como você iniciaria esse projeto.
